

Project Review 1 Report

Problem Definition, Requirement Analysis & Database Design

Project Title:

Movie Recommendation System using Graph-Based NoSQL Database (Neo4j)

Domain:

Recommendation Systems / Graph Databases

Chosen NoSQL Database:

Neo4j (Graph Database)

Team Members:

- Harshit Pandey (24BCT0173)
- Manik Chauhan (24BCE2063)
- Meet Kareliya (24BCE2053)

1. Problem Statement, Objectives & Scope

1.1 Problem Statement

Traditional relational database systems struggle to efficiently model the highly interconnected, many-to-many relationships that exist between users, movies, genres, actors, directors, and ratings in a movie recommendation platform. Queries such as "find movies liked by users similar to me" or "recommend movies based on shared actors and genres with movies I have already rated highly" require multiple expensive JOIN operations in an RDBMS, which do not scale well as the dataset grows. There is a need for a system that can store, query, and traverse these complex relationships efficiently to generate accurate, real-time personalized movie recommendations for users.

1.2 Objectives

- To design and implement a movie recommendation system backed by a NoSQL graph database that naturally represents relationships between users, movies, genres, actors, and directors.
- To enable efficient traversal-based queries for collaborative filtering (users who liked similar movies) and content-based filtering (movies with shared genres/actors/directors).
- To allow users to rate and review movies, and to generate personalized top-N movie recommendations based on their rating history and network of similar users.
- To evaluate the performance and scalability benefits of a graph-based NoSQL model over a traditional relational approach for recommendation-style queries.
- To build a working prototype with a sample dataset demonstrating collaborative and content-based recommendation logic using graph traversal queries (Cypher).

1.3 Scope

In Scope:

- User registration profile, movie catalog, genre/actor/director metadata, and a rating/review system.
- Graph-based storage of relationships: User–RATED→Movie, Movie–BELONGS_TO→Genre, Movie–ACTED_IN(inverse)→Actor, Movie–DIRECTED_BY→Director, User–FOLLOWS→User.
- Recommendation generation via two approaches: (a) Collaborative filtering using shared ratings between users, and (b) Content-based filtering using shared genres/actors/directors.
- A sample dataset of at least 15–20 movies, 10 users, and associated ratings for prototype demonstration.

Out of Scope (for this phase):

- Real-time streaming ingestion of ratings from external platforms (e.g., IMDb, TMDb live APIs).
- Deep-learning based recommendation models (e.g., neural collaborative filtering) — reserved for future extension.
- Payment, subscription, or video-streaming/playback functionality.

2. Requirement Specification

2.1 Functional Requirements

ID	Requirement	Description
FR-1	User Management	Create, update, and view user profiles.
FR-2	Movie Catalog	Store and browse movies with metadata (title, year, genre, cast, director, plot).
FR-3	Rating & Review	Users can rate movies (1–5) and optionally add a text review.
FR-4	Collaborative Recommendation	Recommend movies liked by other users with similar rating patterns.
FR-5	Content-Based Recommendation	Recommend movies sharing genres, actors, or directors with movies the user rated highly.
FR-6	Search & Filter	Search movies by title, genre, actor, or director.
FR-7	Social Graph	Users can follow other users; recommendations can be influenced by followed users' ratings.
FR-8	Top-N Recommendations	Generate a ranked list of top-N recommended movies per user on demand.

2.2 Non-Functional Requirements

ID	Requirement	Description
NFR-1	Performance	Recommendation queries (graph traversal up to 2–3 hops) should return within 1–2 seconds for the sample dataset.
NFR-2	Scalability	Schema should support scaling to thousands of users and movies without structural redesign.
NFR-3	Flexibility	Schema-optional graph model should allow easy addition of new node types (e.g., Language, Awards) later.
NFR-4	Usability	Query interface / API should return recommendations in a simple, consumable format (JSON).
NFR-5	Data Integrity	Relationship properties (e.g., rating value, timestamp) must be consistently maintained.

2.3 Key Stakeholders

- End Users — rate movies and receive personalized recommendations.
- Database/Backend Team — design and maintain the graph schema and query logic.
- Project Evaluators — assess correctness of schema design and recommendation query results.

3. Selection & Justification of NoSQL Database

Three major categories of NoSQL databases were evaluated against the requirements of the Movie Recommendation System: a document store (MongoDB), a wide-column store (Cassandra), and a graph database (Neo4j). The comparison below focuses on how naturally each model supports relationship-heavy recommendation queries.

Criterion	MongoDB (Document)	Cassandra (Wide-Column)	Neo4j (Graph)
Relationship modeling	Relationships modeled via embedding/referencing; multi-hop traversal needs application-side joins.	Optimized for write-heavy, denormalized tables; poor at ad-hoc relationship traversal.	Relationships are first-class citizens (edges); native multi-hop traversal.
Recommendation queries	Requires multiple queries + aggregation pipeline for "similar user" logic.	Requires pre-computed query tables per access pattern; inflexible for exploratory queries.	Single Cypher query can traverse User-RATED-Movie-RATE D-User patterns efficiently (collaborative filtering).
Schema flexibility	Flexible JSON documents; good for catalog metadata.	Flexible column families, but query patterns must be known in advance.	Schema-optional property graph; new node/relationship types added without migration.
Query performance at scale	Degrades with deep relationship traversal (multiple \$lookup joins).	Excellent for high-volume writes/reads on known keys; weak for graph-like traversal.	Index-free adjacency keeps multi-hop traversal performance largely independent of total graph size.
Best fit for this project	Good secondary fit for movie catalog/document metadata.	Not well suited — access patterns are exploratory, not fixed.	Best fit — recommendation is inherently a graph traversal problem.

3.1 Final Selection: Neo4j (Graph Database)

Neo4j has been selected as the NoSQL database for this project. The core justification is that a movie recommendation engine is fundamentally a network of relationships — users connected to movies through ratings, movies connected to each other through shared genres/actors/directors, and users connected to other users through social/behavioral similarity. Graph databases store these relationships as first-class entities with **index-free adjacency**, allowing traversal queries (e.g., "find movies rated highly by users who also liked the movies I liked") to be expressed naturally in a few lines of Cypher and executed efficiently, regardless of overall graph size. This directly satisfies FR-4, FR-5, FR-7 and NFR-1/NFR-3 identified in Section 2.

4. Database Design — Graph Schema

4.1 Node (Entity) Types

Node Label	Key Properties
:User	userId (unique), name, email, age, joinDate
:Movie	movieId (unique), title, releaseYear, duration, language, plotSummary
:Genre	name (unique) e.g. Action, Drama, Sci-Fi
:Person	personId (unique), name, dob — used for both actors and directors via relationship type

4.2 Relationship (Edge) Types

Relationship	Direction	Properties	Meaning
RATED	(User)-[:RATED]->(Movie)	score (1–5), timestamp, reviewText	A user's rating/review of a movie
BELONGS_TO	(Movie)-[:BELONGS_TO]->(Genre)	—	Genre classification of a movie
ACTED_IN	(Person)-[:ACTED_IN]->(Movie)	roleName	An actor's role in a movie
DIRECTED	(Person)-[:DIRECTED]->(Movie)	—	Director of a movie
FOLLOWS	(User)-[:FOLLOWS]->(User)	since	Social connection between users
SIMILAR_TO	(Movie)-[:SIMILAR_TO]->(Movie)	score	Precomputed content similarity (optional, derived)

4.3 Conceptual Graph Schema Diagram (textual)

```
(:User)-[:RATED {score, timestamp}]->(:Movie)
(:User)-[:FOLLOWS {since}]->(:User)
(:Movie)-[:BELONGS_TO]->(:Genre)
(:Person)-[:ACTED_IN {roleName}]->(:Movie)
(:Person)-[:DIRECTED]->(:Movie)
(:Movie)-[:SIMILAR_TO {score}]->(:Movie)
```

4.4 ER-to-Graph Mapping Rationale

- Entities → Nodes: Each strong entity in the conceptual ER model (User, Movie, Genre, Person) becomes a node label in the property graph.
- Relationships (incl. attributed ones) → Edges with properties: The RATED relationship, which in an ER model would need an associative/junction entity (since it carries score, timestamp, reviewText), becomes a single first-class edge with properties in Neo4j — eliminating the extra junction table required in a relational design.
- Many-to-Many relationships (User–Movie via ratings, Movie–Genre, Person–Movie) map directly to graph edges without needing bridge tables, reducing JOIN overhead for traversal-heavy recommendation queries.
- Person is modeled as a single node label (rather than separate Actor/Director entities) with the relationship type (ACTED_IN vs DIRECTED) capturing the role — avoiding data duplication when a person is both an actor and a director.

4.5 Sample Cypher Schema Creation (Constraints & Indexes)

```
CREATE CONSTRAINT user_id IF NOT EXISTS FOR (u:User) REQUIRE u.userId IS UNIQUE;
CREATE CONSTRAINT movie_id IF NOT EXISTS FOR (m:Movie) REQUIRE m.movieId IS UNIQUE;
CREATE CONSTRAINT genre_name IF NOT EXISTS FOR (g:Genre) REQUIRE g.name IS UNIQUE;
CREATE CONSTRAINT person_id IF NOT EXISTS FOR (p:Person) REQUIRE p.personId IS
UNIQUE;
CREATE INDEX movie_title IF NOT EXISTS FOR (m:Movie) ON (m.title);
```

5. Sample Dataset Design

5.1 Sample Users

userId	name	age	joinDate
U1	Aditi Sharma	24	2024-01-15
U2	Rahul Verma	29	2024-02-03
U3	Sneha Iyer	21	2024-03-10
U4	Karthik Raja	27	2024-04-22

5.2 Sample Movies

movieId	title	year	genres
M1	Inception	2010	Sci-Fi, Thriller
M2	The Shawshank Redemption	1994	Drama
M3	Interstellar	2014	Sci-Fi, Drama
M4	The Dark Knight	2008	Action, Crime
M5	3 Idiots	2009	Comedy, Drama

5.3 Sample Ratings

user	movie	score	timestamp
U1	M1	5	2024-05-01
U1	M3	4	2024-05-04
U2	M1	4	2024-05-02
U2	M4	5	2024-05-05
U3	M2	5	2024-05-03
U4	M3	5	2024-05-06

5.4 Sample Cypher Data Insertion

```
// Create users
CREATE (:User {userId:'U1', name:'Aditi Sharma', age:24,
joinDate:date('2024-01-15')});
CREATE (:User {userId:'U2', name:'Rahul Verma', age:29,
joinDate:date('2024-02-03')});

// Create movies and genres
CREATE (:Movie {movieId:'M1', title:'Inception', releaseYear:2010});
MERGE (g:Genre {name:'Sci-Fi'});
MATCH (m:Movie {movieId:'M1'}), (g:Genre {name:'Sci-Fi'})
CREATE (m)-[:BELONGS_TO]->(g);

// Create a rating edge
MATCH (u:User {userId:'U1'}), (m:Movie {movieId:'M1'})
CREATE (u)-[:RATED {score:5, timestamp:datetime('2024-05-01T10:00:00')}]>(m);
```

5.5 Sample Recommendation Query (Collaborative Filtering)

```
// Recommend movies to U1 based on users who rated the same movies highly
MATCH (target:User {userId:'U1'})-[:RATED]->(m:Movie)<-[:RATED]-(other:User)
MATCH (other)-[:RATED]->(rec:Movie)
WHERE NOT (target)-[:RATED]->(rec) AND r.score >= 4
RETURN rec.title AS recommendation, COUNT(*) AS strength
ORDER BY strength DESC
LIMIT 5;
```